

Clustering. Hierarchical Clustering

Modelització Estadística

Grau en Intel·ligència Artificial

Course 2024-25



Outline

- 1 Introduction
- 2 Proximity
- 3 Hierarchical Clustering Basics
- 4 R
- 5 Evaluating Clustering Results
- 6 Hopkins Statistic
- 7 References

Why clustering?

Principal Component Analysis (PCA) utility

- **To reduce the dimensionality**
- To visualize the data with a **minimum lost of information**
- **Exploratory tool** to unveil the hidden structural factors present in data.
- **To represent the reality**, “to see” the overall pattern of data

Problem

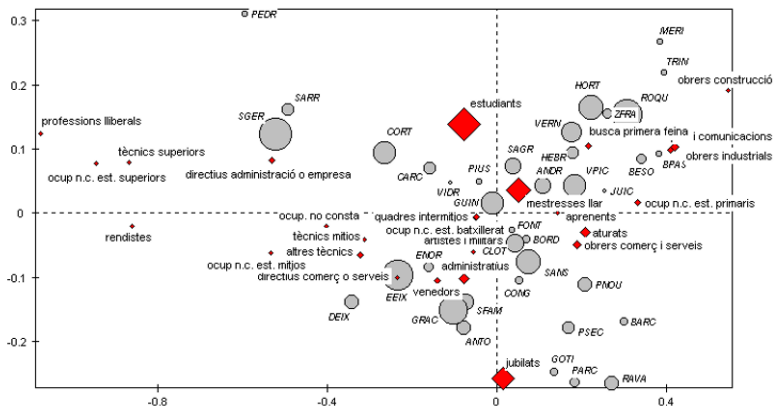
Reality is too complex, we need to simplify it. We need to detect *singular* groups of individuals: **clustering**

Example

- **Marketing**: Customer groups for targeted marketing
- **Astronomy**: Groups of similar stars and galaxies
- **Earth-quakes**: Epicenters should be clustered along continent faults
- **Genomics**: Groups of genes with similar expressions

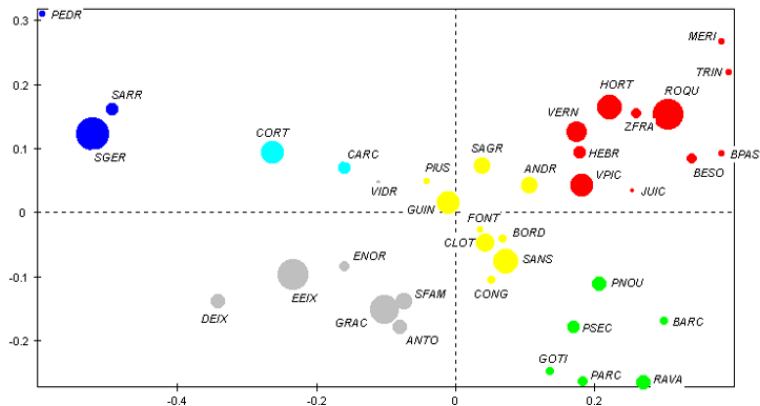
Example (I)

Characteristics of the Barcelona's neighborhoods.



Example (II)

Clusterization of the Barcelona's neighborhoods.



Types of clustering algorithms

- **Bottom-up (hierarchical)**

- Single linkage
- Average linkage
- Ward
- ...

- **Direct Partitioning**

- K-means
- K-medoids
- K-medians
- ...

- **Probabilistic**

- EM
- ...

What are Proximity Measures?

What are Proximity Measures?

Proximity measures quantify how **similar** or **different** objects are in a dataset. They are critical for clustering as they determine how objects are grouped.

- Used to build a **dissimilarity matrix**.
- Choice of measure depends on the **nature of the data**. E.g:
 - numerical
 - categorical

Proximity measures for numerical data

Common Distance Metrics for Numerical Data

- **Euclidean distance:** Straight-line distance between two points in space.

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

- **Manhattan Distance:** Sum of absolute differences across dimensions.

$$d(x, y) = \sum_{i=1}^n |x_i - y_i|$$

- **Minkowski distance:** Generalization of Euclidean and Manhattan.

$$d(x, y) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{1/p}$$

- **Correlation distance:** Measures dissimilarity based on the correlation between variables (e.g. Mahalanobis distance)

Proximity measures for numerical data. Example

Graphical representation of **Minkowski distance** depending on p :



Example

Suppose you have two objects:

$$x = (2, 3), \quad y = (4, 7)$$

Using **Euclidean** distance:

$$d(x, y) = \sqrt{(2 - 4)^2 + (3 - 7)^2} = \sqrt{4 + 16} = \sqrt{20} \approx 4.47$$

Using **Manhattan** distance:

$$d(x, y) = |2 - 4| + |3 - 7| = 2 + 4 = 6$$

Proximity measures for numerical data. Tips

Considerations When Choosing a Metric

- **Scale of Variables:** Standardize variables if they have different units or ranges.
- **Data Type:** Use specific measures for categorical or mixed data types.
- **Domain Knowledge:** Select a metric that reflects meaningful relationships in your dataset.

Standardizing Variables

When variables have different scales, large-scale variables dominate the distance calculation. Standardize variables to have zero mean and unit variance:

$$z = \frac{x - \mu}{\sigma}$$

Proximity measures for categorical data

Challenges with Categorical Data

Clustering categorical data requires **specialized similarity measures** because traditional distance metrics like Euclidean distance are not meaningful.

Proximity measures for categorical data

Common Measures for Categorical Data

- **Matching coefficient (Simple matching index):**

$$S(x, y) = \frac{M}{p}$$

- M : Number of matching attributes.
- p : Total number of attributes.

- **Jaccard index:**

$$J(x, y) = \frac{M}{M + U}$$

- Ignores cases where both attributes are 0.
- M : Matching attributes where both are 1.
- U : Non-matching attributes.

- **Hamming distance:**

$$d(x, y) = \sum_{i=1}^p \mathbf{1}(x_i \neq y_i)$$

- Counts the number of mismatches between two categorical objects.

Proximity measures for categorical data. Example

Example

Suppose we have two categorical objects:

$$x = (\text{Red}, \text{Cat}, \text{High}), \quad y = (\text{Blue}, \text{Dog}, \text{High})$$

Matching coefficient:

$$S(x, y) = \frac{1}{3} = 0.33 \quad (1 \text{ match out of 3 attributes})$$

Hamming distance:

$$d(x, y) = 2 \quad (2 \text{ mismatched attributes})$$

Proximity measures for categorical data. Tips

Practical Considerations

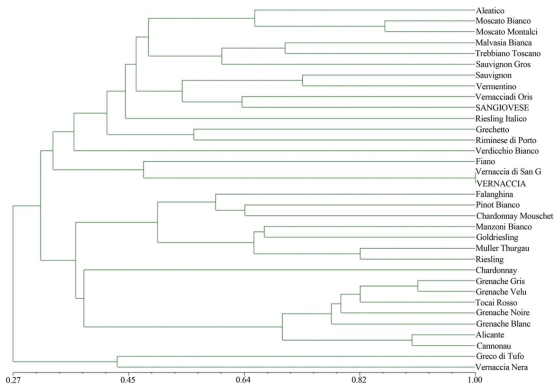
- **Binary Attributes:** Use Jaccard index for sparse data (e.g., presence/absence).
- **Mixed Data:** For datasets with both numerical and categorical variables, use techniques like **Gower's distance**.
- Preprocess data to ensure **uniform representation** of categorical variables (e.g., encoding as factors or binary indicators).

What is Hierarchical Clustering?

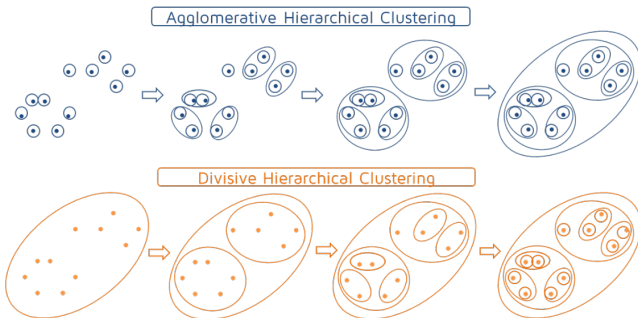
Definition of Hierarchical Clustering

Hierarchical clustering is a method of grouping objects into a tree-like structure, where clusters are formed at multiple levels of granularity.

- The result is often visualized using a **dendrogram**.
- It provides a hierarchy of clusters, allowing users to explore groupings at **different levels of detail**.



Types of Hierarchical Clustering



Two Types of Hierarchical Clustering

- **Agglomerative (Bottom-Up):**

- Starts with each object as its own cluster.
- Clusters are merged iteratively until one large cluster is formed.

- **Divisive (Top-Down):**

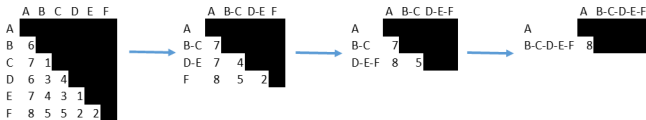
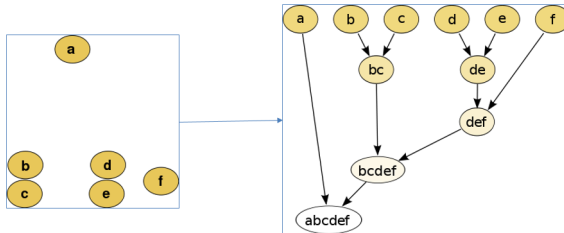
- Starts with all objects in a single cluster.
- Splits are performed iteratively until all objects are individual clusters.

Types of Hierarchical Clustering. Examples of agglomerative

Example

In an **agglomerative** clustering of animals:

- Start with each animal as its own cluster.
- Merge the most similar animals, such as "dog" and "wolf."
- Continue merging until all animals belong to one cluster.



Pros and cons of Hierarchical Clustering?

Advantages of Hierarchical Clustering

- **No need to predefine the number of clusters.**
- Can work with both **numerical and categorical data**.
- Captures **hierarchical relationships** in the data.

Limitations of Hierarchical Clustering

- **Computationally intensive** for large datasets.
- **Sensitive to noise and outliers**, which can distort the hierarchy.
- **Difficult to revise decisions** once clusters are merged or split.

Key Concepts: Dissimilarity Matrix

The Dissimilarity Matrix

The **dissimilarity matrix** is a table showing pairwise distances between all objects in the dataset.

- It is the **foundation** of hierarchical clustering.
- Common distance metrics:
 - **Euclidean** distance: Straight-line distance between points.
 - **Manhattan** distance: Sum of absolute differences.
- A symmetric matrix where $D(i, j)$ is the distance between objects i and j , with the following **properties**:
 - $D(i, j) = 0 \leftrightarrow i = j$
 - $D(i, j) = D(j, i)$
 - $D(i, j) \leq D(i, k) + D(k, j)$ (*triangular inequality*)

Key Concepts: Linkage Criteria

Linkage criteria determine how distances between clusters are calculated.

Linkage Criteria

- **Single Linkage:**

$$d(A, B) = \min\{d(a, b) : a \in A, b \in B\}$$

Uses the smallest distance between two points in different clusters.

- **Complete Linkage:**

$$d(A, B) = \max\{d(a, b) : a \in A, b \in B\}$$

Uses the largest distance between two points in different clusters.

- **Average Linkage:**

$$d(A, B) = \frac{1}{|A| \cdot |B|} \sum_{a \in A, b \in B} d(a, b)$$

Uses the average distance between all points in the clusters.

- **Ward's Method:** Minimizes the increase in variance when two clusters are merged.

Key Concepts: Linkage Criteria. Example

Example

Consider three objects: $A = (1, 1)$, $B = (4, 1)$, and $C = (4, 5)$.

The dissimilarity matrix (Euclidean distance):

$$D = \begin{bmatrix} 0 & 3 & 5 \\ 3 & 0 & 4 \\ 5 & 4 & 0 \end{bmatrix}$$

If A and B are in a cluster:

- **Single Linkage** → Distance to $C = 4$
- **Complete Linkage** → Distance to $C = 5$
- **Average Linkage** → Distance to $C = \frac{4+5}{2} = 4.5$

Key Concepts: Inertia

Inertia

Inertia is a measure of the **heterogeneity** of a cluster. Given a partition into Q groups formed by I_q elements in a K -dimensional space (K variables):

- **Total inertia.** Total variability (independent of clusterization).

$$I_T = \sum_{k=1}^K \sum_{q=1}^Q \sum_{i=1}^{I_q} (y_{iqk} - \bar{y}_k)^2$$

- **Between-groups inertia.** Distance from cluster center to global one.

$$I_B = \sum_{k=1}^K \sum_{q=1}^Q |I_q| (\bar{y}_{qk} - \bar{y}_k)^2$$

- **Within-group inertia.** Distance from each point to its cluster center.

$$I_W = \sum_{k=1}^K \sum_{q=1}^Q \sum_{i=1}^{I_q} (y_{iqk} - \bar{y}_{qk})^2$$

Relationship between inertias

$$I_T = I_B + I_W$$

Key Concepts: Inertia and quality of partition

Quality of a partition

The **quality of a partition** can be measured as the percentage of variability explained by the clusters, that is:

$$\text{Explained variability} = \frac{I_B}{I_T}$$

The percentage of explained variability always grows as the number of clusters increases:

- **Higher** number of clusters $\leftrightarrow$ **Higher** explained variability $\leftrightarrow$ **Higher** between-group inertia
- **Less** number of clusters $\leftrightarrow$ **Less** explained variability $\leftrightarrow$ **Less** between-group inertia

Key Concepts: Ward's Method

Ward's Method

- We start with Q clusters
- An ideal partition with $(Q - 1)$ clusters is desired
- Let p (with center of gravity g_p and size l_p) and q (with center of gravity g_q and size l_q) be two clusters. The **increase in within-group inertia** is quantified by:

$$\Delta(p, q) = \frac{l_p l_q}{l_p + l_q} d^2(p, q)$$

- The Ward's criterion of putting together those clusters such that they **minimize the change in within-group inertia**.

This criterion favors grouping:

- The **nearby** clusters
- The **small** clusters

Key Concepts: Linkage Criteria. Properties

Linkage Criteria. Properties

- **Single Linkage**. This produces an undesirable phenomenon called chaining effect, which entails a **violation of the property of compactness**. This method produces clusters with larger diameters (distance between their points).
- **Complete Linkage**. It produces clusters with small diameters but which **violate the property of connectivity**
- **Average Linkage**. It represents a middle ground between the 2 previous solutions. However, the solution **depends on the selected scale**: e.g, this method does not provide the same solution on log-transformed variables
- **Ward's Method**. It is the **preferred criterion** to avoid some of the abovementioned drawbacks.

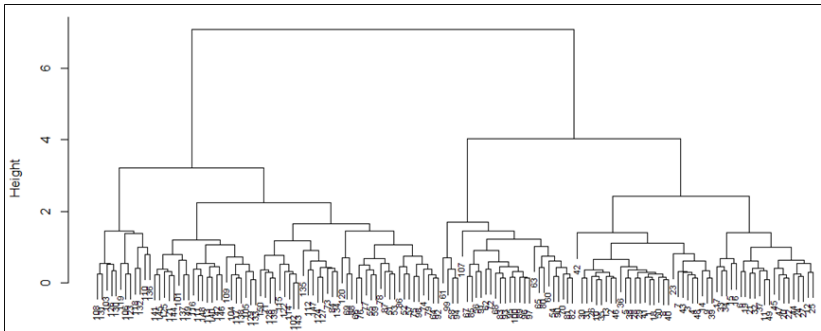
Dendrograms

Dendrograms

A **dendrogram** is a tree-like diagram that shows the clustering process.

- Each **horizontal line** represents a merge of clusters.
- The **height** of the line indicates the dissimilarity between the merged clusters.
- Cutting the dendrogram at a certain height determines the **number of clusters**.

Example of dendrogram



Interpreting a Dendrogram

- Objects connected at **lower heights** are **more similar**.
- Choose a **threshold to create clusters** by cutting the dendrogram at that level.

Step-by-Step Process in Hierarchical Clustering

Overview of Steps

To perform hierarchical clustering, follow these key steps:

- 1 **Choose a Distance Metric:** Calculate pairwise distances using metrics such as Euclidean, Manhattan, etc.
- 2 **Build a Dissimilarity Matrix:** Represent the calculated distances between all pairs of objects.
- 3 **Select a Linkage Criterion:** Decide how to measure the distance between clusters (e.g., single, complete, average linkage, Ward's method).
- 4 **Construct the Dendrogram:** Iteratively merge or split clusters and visualize the hierarchy.
- 5 **Determine the Number of Clusters:** Cut the dendrogram at a desired height to create clusters.

Hierarchical clustering. Tips

Tips for Practical Applications

- **Standardize variables** if they are on different scales before calculating distances.
- Choose a linkage method that aligns with the goals of your analysis:
 - **Single linkage**: Captures chain-like clusters but can suffer from chaining effects.
 - **Complete linkage**: Creates compact clusters but may ignore small clusters.
 - **Ward's method**: Optimizes cluster homogeneity and is ideal for most use cases.
- Use **dendrogram height** to explore multiple cluster solutions.

Example: Iris dataset

Dataset: Iris Flower Measurements

The `iris` dataset contains measurements of 150 iris flowers from three species (*setosa*, *versicolor*, and *virginica*). Variables include:

- Sepal Length
- Sepal Width
- Petal Length
- Petal Width

Our goal is to group the flowers based on these measurements using hierarchical clustering.

Example: Step 1. Prepare the data

Step 1: Prepare the Data

- **Standardizing** ensures variables contribute equally regardless of their scale.
- After scaling, all variables now have **zero mean and unit variance**.

```
# Load the dataset and remove the species column  
data <- iris[, -5] # Exclude the "Species" column  
  
# Standardize the data  
data_scaled <- scale(data)
```

Data (only first 5 rows):

Sepal.Length	Sepal.Width	Petal.Length	Petal.Width
5.1	3.5	1.4	0.2
4.9	3.0	1.4	0.2
4.7	3.2	1.3	0.2
4.6	3.1	1.5	0.2
5.0	3.6	1.4	0.2

Example: Step 2. Perform Hierarchical Clustering

Step 2: Perform Hierarchical Clustering

- **Distance Metric:** **Euclidean distance** measures the straight-line distance between data points.
- **Linkage Criterion:** **Ward's method** minimizes within-cluster variance during merges.

```
# Compute the distance matrix
```

```
dist_matrix <- dist(data_scaled, method = "euclidean")
```

```
# Apply hierarchical clustering with Ward's method
```

```
hc <- hclust(dist_matrix, method = "ward.D2")
```

Distance matrix (only first 5 flowers):

	1	2	3	4	5
1	0.000	1.172	0.843	1.100	0.259
2	1.172	0.000	0.522	0.433	1.382
3	0.843	0.522	0.000	0.283	0.988
4	1.100	0.433	0.283	0.000	1.246
5	0.259	1.382	0.988	1.246	0.000

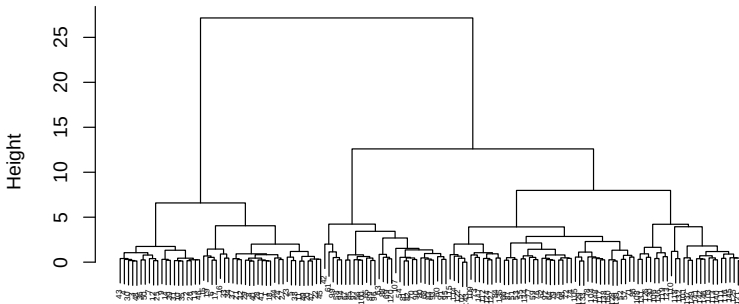
Example: Step 3. Visualize the Dendrogram

Step 3: Visualize the Dendrogram

- The dendrogram shows how flowers are **grouped at different levels**.
- The **height** of the horizontal lines indicates the **dissimilarity** at which clusters are merged.

```
plot(hc, main = "Dendrogram of Iris Data", xlab = "", sub = "", cex=0.4
```

Dendrogram of Iris Data



Example: Step 4. Interpretation

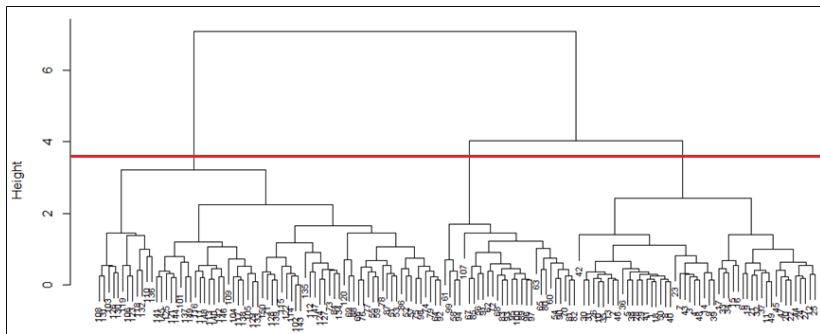
Key Insights from the Example

- **Interpreting the Dendrogram:**
 - Objects connected at lower heights are more similar.
 - The number of clusters depends on where the dendrogram is cut.
- **Performance with Known Labels:** Comparing clusters with the actual species shows how well the clustering captures the structure of the data.
- **Scalability:** This method works well for small to medium-sized datasets but may be computationally expensive for larger datasets.

Example: Step 5. Number of clusters

How to determine the number of clusters

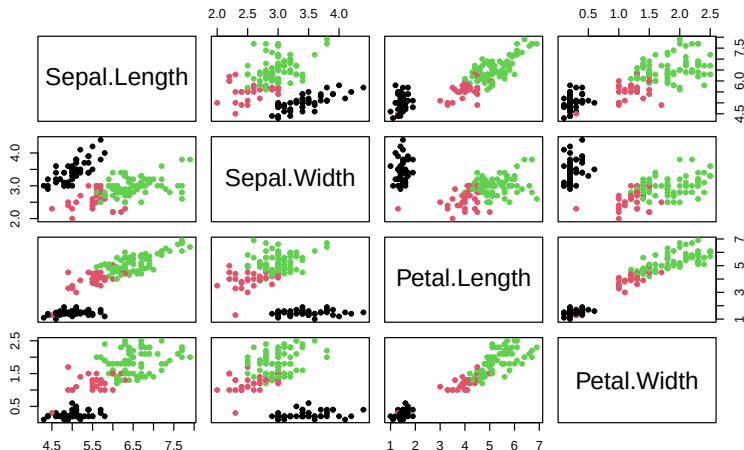
By **visual inspection**, we can determine the total number of clusters



In this case, the **number of clusters could be 3**.

Example: Step 6. Visualize clusters

```
ct <- cutree(hc,3)
pairs(data, col = ct, pch = 19, cex = 0.8)
```



Quality of Clustering

Why Evaluate Clustering Quality?

Evaluating clustering quality ensures that the clusters accurately reflect the data's structure.

- How well do the clusters fit the data?
- Are the distances between clusters meaningful?
- How consistent is the dendrogram with the original distances?

Cophenetic Correlation

Cophenetic Correlation Coefficient

The **cophenetic correlation coefficient** measures how faithfully the dendrogram preserves the original pairwise distances.

- Compare the original distance matrix with the cophenetic distance matrix (distances from the dendrogram).
- The coefficient is calculated as the correlation between these two matrices:

$$\text{Cophenetic Correlation} = \text{cor}(\text{cophenetic}(hc), \text{dist}(data))$$

- **Values close to 1** indicate a good fit between the dendrogram and the original data.

Example

Evaluate clustering quality for the iris dataset using cophenetic correlation:

```
# Compute the distance matrix
dist_matrix <- dist(scale(iris[, -5]), method = "euclidean")

# Perform hierarchical clustering
hc <- hclust(dist_matrix, method = "ward.D2")

# Calculate the cophenetic correlation
cophenetic_cor <- cor(cophenetic(hc), dist_matrix)

# Print the result
print(paste("Cophenetic Correlation:", round(cophenetic_cor, 2)))
```

[1] "Cophenetic Correlation: 0.82"

Example

A **cophenetic correlation of 0.75 or higher** generally indicates good clustering quality.

Tips for Improving Clustering Quality

Tips for Improving Clustering Quality

- **Standardize the Data:** Ensure all variables contribute equally to distance calculations.
- **Choose the Right Distance Metric:** Different metrics capture different relationships.
- **Optimize Linkage Criteria:** Experiment with methods like Ward's or average linkage for better fits.

Determining the Number of Clusters. Dendrogram

Using the Dendrogram

A dendrogram provides a visual guide for choosing the number of clusters:

- Cut the dendrogram horizontally at a meaningful height.
- The number of vertical lines cut by the horizontal line determines the number of clusters.
- Look for large jumps in height, which indicate significant merges.

Example

Using the `iris` dataset, the red line cuts the dendrogram into **three clusters**. Adjust the height to explore different numbers of clusters.

Determining the Number of Clusters. Dendrogram

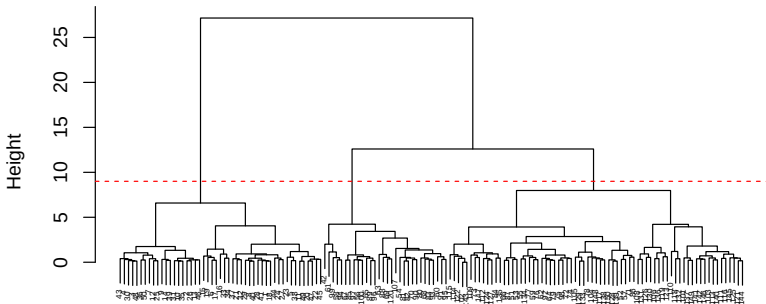
```
# Plot the dendrogram
```

```
plot(hc, main = "Dendrogram of Iris Data", xlab = "", sub = "", cex = 0
```

```
# Draw a horizontal line to cut the dendrogram
```

```
abline(h = 9, col = "red", lty = 2)
```

Dendrogram of Iris Data



Determining the Number of Clusters. Elbow method

The Elbow Method

The **elbow method** identifies the optimal number of clusters by analyzing within-cluster variance:

- For hierarchical clustering, compute one of the following measures:
 - The total within-cluster inertia
 - The between-cluster inertia
 - The percentage of variability explained
- Plot the selected measure against the number of clusters.
- Look for the **"elbow"**, where the rate of decrease/increase slows significantly.

Determining the Number of Clusters. Example

Function to calculate **Within-cluster inertia**:

```
# Within-cluster inertia for different numbers of clusters
wss <- sapply(1:10, function(k) {
  clusters <- cutree(hc, k)
  sum(sapply(unique(clusters), function(cluster) {
    indices <- which(clusters == cluster)
    cluster_data <- data[indices, ]
    sum(scale(cluster_data, scale = FALSE)^2)
  })))
})
```

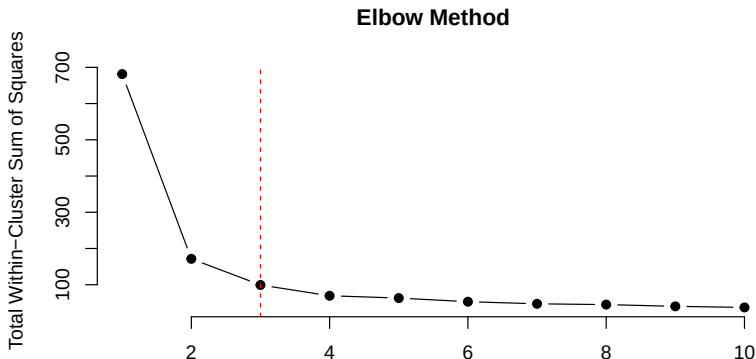
Example

Evaluate the optimal number of clusters using the **elbow method**. The red line (see next slide) shows the optimal number of clusters ($k = 3$).

Determining the Number of Clusters. Example

```
# Plot the elbow method
```

```
plot(1:10, wss, type = "b", pch = 19, frame = FALSE,  
     xlab = "Number of Clusters",  
     ylab = "Total Within-Cluster Sum of Squares",  
     main = "Elbow Method")  
abline(v = 3, lty = 2, col = "red")
```



Tips for Determining the Number of Clusters

Tips for Deciding the Number of Clusters

- **Combine methods:** Use both the dendrogram and elbow method for robust decisions.
- Consider **domain knowledge:** Ensure the chosen number aligns with meaningful groupings in your dataset.
- Experiment with different cluster numbers to validate **stability**.

What is the Hopkins Statistic?

Purpose of the Hopkins Statistic

The **Hopkins statistic** is a measure of **clustering tendency**, which determines whether a dataset contains meaningful clusters or is uniformly distributed.

- Values **close to 1** indicate strong **clustering tendency**.
- Values **close to 0.5** suggest uniform or **random distribution**.
- It is especially useful before applying clustering algorithms to **avoid wasting resources on unsuitable data**.

How Hopkins Statistic Works?

How Hopkins Statistic Works

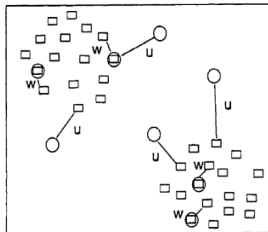
The **Hopkins statistic** is calculated as:

$$H = \frac{\sum_{i=1}^n u_i}{\sum_{i=1}^n w_i + \sum_{i=1}^n u_i}$$

- w_i : Nearest-neighbor distances from random points in the data to other data points.
- u_i : Nearest-neighbor distances from uniformly distributed synthetic points to the data points.

Steps:

- 1 Randomly sample points from the dataset (n).
- 2 Generate synthetic points uniformly distributed in the data space.
- 3 Compute w_i and u_i distances.
- 4 Calculate H .



Hopkins Statistic. Example

Example

Let's compute the Hopkins statistic for the iris dataset in R using the clustertend package. Interpretation:

- $0.75 < H < 1$: Strong clustering tendency.
- $0.60 < H < 0.75$: Low clustering tendency.
- $0 < H < 0.60$: No clustering tendency.

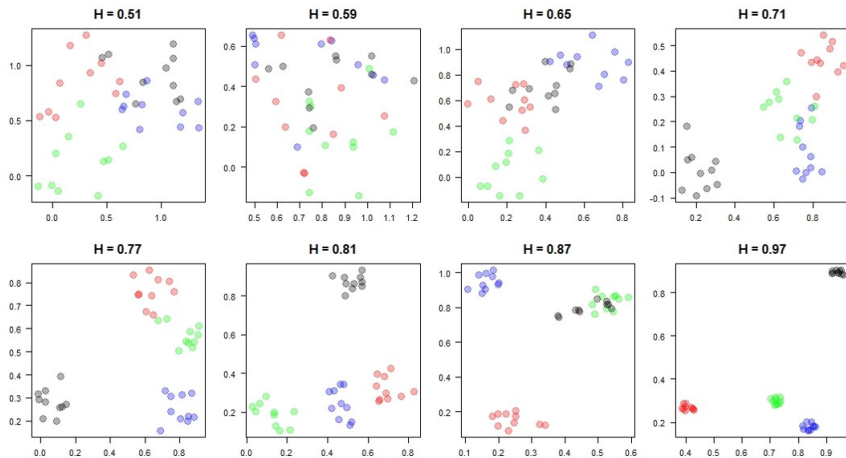
```
# Install and load necessary library  
# install.packages("hopkins")  
library(hopkins)
```

```
# Compute the Hopkins statistic  
set.seed(123)  
hopkins_stat <- hopkins(data)
```

```
# Print the result  
print(paste("Hopkins Statistic:", round(hopkins_stat, 3)))
```

```
## [1] "Hopkins Statistic: 0.999"
```

Example. Different values of Hopkins statistics



Hopkins statistic. Practical considerations

Practical Considerations for Hopkins Statistic

- Works best with **numerical data**. Scale the data if necessary to ensure equal contribution from all variables.
- The sample size (n) of **random points** significantly impacts the result; choose it carefully.
- Hopkins statistic only tests **clustering tendency**, not the number or quality of clusters.

References

- A. Kassambara, Practical guide to cluster analysis in R: Unsupervised machine learning. Sthda, 2017, vol. 1.
- Hastie, T., Tibshirani, R. & Friedman, J. H. The Elements of Statistical Learning Data Mining, Inference, and Prediction. Springer (2009)